N-Body Simulation Project

Milestone 4: Barnes-Hut Force Calculation Vectorisation

Group Report

Hafsa Tanveer	29041
Manal Fatima	29018
Areeba Salman	29075

1 The project in retrosprective

This is the end of a multi-stage project that has covered the whole semester. It should perhaps be useful to look briefly at the previous milestones to get an idea of the context in which the present one will be developed.

Milestone 1: The project started with two separate solutions of the N-body problem. To set a correctness baseline, and to gain an intuitive understanding of the dynamics of the simulation, a direct $O(N^2)$ pairwise force calculation was implemented in Python. At the same time, the Barnes-Hut algorithm in C++ was built which showed the O(N log N) tree-based approach that would be used in all later work.

Milestone 2: The emphasis moved from showing how it was done to measuring its performance and implementing it at low level. It was rewritten for naive Python in order to make it "readable and as an example to test correctness. More importantly, a scalar RISC-V assembly implementation of the naive implementation of computing the force was written. This was the first time that hand-optimized assembly was encountered for this problem set and introduced and established the register conventions, memory layout and loop structure that would be expanded on in subsequent milestones.

Milestone 3: This milestone was set into two tracks. To implement the naive $O(N^2)$ force calculation, first the scalar assembly code of Milestone 2 was expanded to execute the instructions using the RISC-V Vector Extension (RVV). This proved the strip-mining idiom and showed performance advantages of vectorising regular, data parallel tasks. Secondly, the C++ Barnes-Hut implementation was significantly refactored; with the elimination of dynamic memory allocation in favour of flat arrays, the removal of object-oriented abstractions and flattening the octree into a node pool, indexed by integer array elements. This refactoring is needed for a future attempt to implement the tree traversal in scalar assembly; it is also needed to avoid the pitfalls of using recursion and pointer chasing in assembly language.

Milestone 4 (Current): Solving the Barnes-Hut force calculation is the next step, which is addressed in the present work. The scalar assembly implementation of the tree traversal at the end of Milestone 3 is not vectorised because the traversal itself is Data Dependent and Recursive, and hence cannot be vectorised. But the arithmetical kernel which gives force to a set of particles from a node is very well suited to vectorisation. This vectorisation has been done by each group member in their own way and based on different research papers read from the literature. Each implementation is explained separately in the subsequent sections.

2 Hafsa Tanveer (29041) — Approach and Implementation

2.1 Research Summary

Two papers shaped the direction of this implementation, and both are worth discussing in some detail because they approach the vectorisation problem from quite different angles.

Makino (1990) — Vectorization of a Treecode. Makino's paper [1] is one of the earliest serious treatments of how to get vector performance out of a Barnes-Hut simulation. The core difficulty he identifies is something that becomes obvious the moment you look at the scalar traversal code: the tree walk for particle i produces an interaction list whose length and contents are completely unpredictable until runtime. That irregularity kills naive vectorisation because you cannot feed a SIMD unit a ragged, particle-specific list without some preprocessing step.

Makino's solution is conceptually clean. Rather than trying to vectorise the traversal itself—which he correctly identifies as hopeless in the general case—he separates the computation into two phases. In the first phase the tree walk is performed in the usual scalar fashion, but instead of immediately accumulating forces it *collects* the interaction partners into an explicit list. In the second phase that list is handed to a purely arithmetic kernel that computes all pairwise force contributions in a tight, branch-free loop. Because the second phase contains no conditionals and no pointer chasing, it can be vectorised straightforwardly.

The key insight that translated directly into code here is that the interaction list is not a nuisance to be worked around but rather the *interface* between the irregular tree traversal and the regular arithmetic. Once that separation is made explicit in the design, the vectorisation almost writes itself. The scalar reference code from Milestone 3 accumulated forces inside the traversal, which meant a refactor was needed to pull the kernel out; Makino's framing made it obvious where to draw that boundary.

Warren and Salmon (1993) — A Parallel Hashed Oct-tree N-body Algorithm. Warren and Salmon [2] are primarily concerned with distributed-memory parallelism, but their paper contains an important discussion of spatial locality and particle ordering that is directly relevant here. They observe that the traversal cost per particle is not uniform: particles that are spatially close to one another tend to produce similar interaction lists, whereas particles on opposite sides of the simulation volume produce almost entirely disjoint lists. If particles are laid out in memory in an order that reflects their spatial arrangement, then adjacent particles in the array will tend to visit the same tree nodes during their walks, which improves cache reuse substantially.

The ordering they propose is based on the Morton Z-curve: each particle is assigned a key by interleaving the binary representations of its three normalised coordinates, and the particles are then sorted by these keys before the tree walk begins. The `sort_bodies_morton()` function in the codebase implements exactly this, so the locality benefit described by Warren and Salmon is already present. What their paper contributed to the thinking here was a justification for *why* that pre-sort matters: it is not just a cosmetic reorganisation but a deliberate cache-locality strategy that complements the vectorised force kernel. The Morton sort test confirmed a mean adjacent distance improvement of 2.05× (before: 3.598×10^{12} m, after: 1.757×10^{12} m), which matches the locality improvement Warren and Salmon describe in their paper.

Taken together, both papers point toward the same architecture: sort particles along a space-filling curve to improve locality, walk the tree per-particle to build interaction lists, then process each list with a vectorised arithmetic kernel. That is precisely the structure of the implementation described below.

2.2 Implementation Design

The scalar Milestone 3 code computes forces through a single recursive function, `calculate_force_bh`, which simultaneously traverses the tree and accumulates the gravitational acceleration for one body. The recursion exits at two places: when a leaf containing the query particle is reached (self-interaction, skipped), and when the Barnes-Hut opening criterion $s/d < \theta$ is satisfied, at which point the node's centre-of-mass is treated as a point mass and its contribution is added directly to the running acceleration total. Traversal logic, criterion check, and arithmetic are all tangled together inside a single function.

The vectorised implementation untangles these two concerns. The traversal is preserved almost unchanged as `get_interaction_list`, which performs an iterative depth-first walk using an explicit stack (avoiding Python's recursion limit) and returns a plain list of node indices that passed the opening criterion. Once that list exists the arithmetic is entirely delegated to `compute_force_numpy`, which operates on NumPy arrays with no Python-level loop.

Scalar inner kernel (Milestone 3 reference)

```
def calculate_force_bh(body_idx, node_idx, theta_val):
    if node_mass[node_idx] == 0.0:
        return
    if node_is_leaf[node_idx] and node_particle[node_idx] == body_idx:
        return  # skip self

    dx = node_com_x[node_idx] - px[body_idx]
    dy = node_com_y[node_idx] - py[body_idx]
    dz = node_com_z[node_idx] - pz[body_idx]
    r2 = dx*dx + dy*dy + dz*dz
    r  = math.sqrt(r2)

    if node_is_leaf[node_idx] or (r > 0.0 and
                                  node_size[node_idx] / r < theta_val):
        r_soft  = math.sqrt(r2 + softening * softening)
        r_cubed = r_soft ** 3
        factor  = G * node_mass[node_idx] / r_cubed
        ax[body_idx] += factor * dx
        ay[body_idx] += factor * dy
        az[body_idx] += factor * dz
    else:
        for k in range(8):
            child = node_child[node_idx * 8 + k]
            if child != -1:
                calculate_force_bh(body_idx, child, theta_val)
```

Listing 1: Scalar force accumulation inside tree traversal

NumPy vectorised kernel

```
def compute_force_numpy(i, ilist_np, px_np, py_np, pz_np,
                        soft, ncx, ncy, ncz, nm):
    # gather displacements for all interaction partners at once
    dx = ncx[ilist_np] - px_np[i]
    dy = ncy[ilist_np] - py_np[i]
```

```
    dz = ncz[ilist_np] - pz_np[i]

    # softened squared distance then inverse cube -- no Python loop
    dist_sq   = dx*dx + dy*dy + dz*dz + soft*soft
    inv_dist3 = dist_sq ** -1.5        # vectorised power over whole array

    f = G * nm[ilist_np] * inv_dist3   # element-wise multiply

    return np.sum(f * dx), np.sum(f * dy), np.sum(f * dz)
```

Listing 2: Vectorised force kernel operating on the interaction list

Outer loop and array preparation

The outer loop in `calculate_forces_bh_vectorized` converts all node centre-of-mass and mass arrays to NumPy arrays *once*, before iterating over particles. This avoids repeated `np.array()` conversions inside the hot loop and ensures that the indexed gather in the kernel above is a true vectorised operation:

```
def calculate_forces_bh_vectorized(root):
    global ax, ay, az

    # convert node arrays once, outside the particle loop
    px_np  = np.array(px);   py_np  = np.array(py);   pz_np  = np.array(pz
        )
    ncx_np = np.array(node_com_x)
    ncy_np = np.array(node_com_y)
    ncz_np = np.array(node_com_z)
    nm_np  = np.array(node_mass)

    for i in range(n):
        ilist = get_interaction_list(i, root, theta)  # scalar traversal
        if ilist:
            ilist_np = np.array(ilist, dtype=np.int32)
            ax[i], ay[i], az[i] = compute_force_numpy(
                i, ilist_np, px_np, py_np, pz_np,
                softening, ncx_np, ncy_np, ncz_np, nm_np)
        else:
            ax[i] = ay[i] = az[i] = 0.0
```

Listing 3: Outer driver: one-time array conversion then per-particle dispatch

What changed and why

Three specific changes account for the entire speed difference between the two versions.

First, the `for node in interaction_list` loop is eliminated entirely. In the scalar code each iteration is a separate Python bytecode dispatch cycle; in the NumPy version the equivalent work is expressed as a single indexed array operation (`ncx_np[ilist_np]`), which NumPy executes in a compiled C loop with no interpreter overhead per element.

Second, the distance computation uses the `** -1.5` power operator applied to a whole array. NumPy lowers this to a vectorised reciprocal-power call operating on all interaction partners simultaneously. The scalar version calls `math.sqrt` once per partner, paying interpreter overhead every time.

Third, `np.sum` is a vectorised horizontal reduction. It accumulates all force contributions in a single compiled pass, whereas the scalar version adds to a running Python float on every loop iteration.

2.3 Correctness Testing

Correctness was verified in two complementary ways: a full automated test suite run before any simulation output is produced, and physical conservation checks over a multi-step run.

Automated test suite results

All six tests in the test suite passed on the 100-body dataset. The results are summarised below.

Mass conservation. The root node accumulated mass of 2.000518×10^{30} kg matched the direct particle sum of 2.000518×10^{30} kg to a relative error of 5.63×10^{-16}, which is at the level of IEEE 754 double-precision rounding and confirms that the tree build accumulates mass without any loss.

All particles found. A depth-first traversal of the completed tree located every particle index exactly once with no duplicates or missing entries. This confirms that `insert_particle` correctly places every body into the octree regardless of its spatial position.

Centre of mass. The tree-computed centre of mass matched the direct weighted average to full double precision on all three axes:

$$x = 5.670615 \times 10^{7} \text{ m},$$
$$y = 1.159580 \times 10^{7} \text{ m},$$
$$z = -1.031701 \times 10^{7} \text{ m}.$$

The agreement to all six printed digits confirms that `compute_mass` propagates the weighted centre-of-mass correctly from leaves to root.

Bounding boxes. Every particle was confirmed to lie within the bounding box of its leaf node on all three axes. This checks that `subdivide` assigns child extents consistently with `get_octant`.

Morton locality. After sorting, the mean adjacent-particle distance fell from 3.598×10^{12} m to 1.757×10^{12} m, a 2.05× improvement. This matches the spatial locality benefit described by Warren and Salmon and confirms that the Morton key computation and sort are working correctly.

Regression (one step). After one complete leapfrog KDK step the positions of the first five bodies were:

Body	x(m)	y(m)	z (m)
0	1.850×10^{11}	-1.801×10^{12}	-3.655×10^{12}
1	1.511×10^{11}	-1.978×10^{10}	-4.698×10^{12}
2	9.689×10^{10}	2.486×10^{11}	-4.618×10^{12}
3	-3.625×10^{11}	-6.152×10^{10}	-4.102×10^{12}
4	-9.834×10^{11}	-1.857×10^{11}	-2.661×10^{12}

No NaN or Inf values were present in any position component after the step, confirming numerical stability for at least one full integration cycle.

Why the scalar and NumPy outputs agree

The vectorised kernel produces accelerations numerically identical to the scalar reference up to floating-point rounding. The tiny discrepancy that does exist (of order 10^{-12} m s^{-2}) is not a bug but an expected consequence of floating-point non-associativity: the scalar code accumulates contributions in traversal order, while `np.sum` uses a pairwise tree reduction internally. Both orderings are correct; they simply round differently at the last few bits.

2.4 Performance

Wall-clock time was measured using `time.perf_counter` around the force calculation call only, excluding tree construction, Morton sort, and diagnostic output. Tests were run on the same machine for both implementations to ensure a fair comparison. The results are shown in Table 1.

Table 1: Force calculation wall-clock time: scalar vs. vectorised

N	Scalar (s)	NumPy (s)	Speedup
100	0.0236	0.0150	1.57×
500	0.2739	0.1669	1.64×
1000	0.8336	0.4158	2.00×
2000	2.0583	0.9918	2.08×

The speedup grows steadily from 1.57× at $N = 100$ to 2.08× at $N = 2000$. This trend makes sense for two reasons. First, the average interaction-list length grows roughly as $O(\log N)$ for a balanced tree with $\theta = 0.5$, so each call to `compute_force_numpy` processes more elements per invocation as N increases. The fixed overhead of converting the interaction list to a NumPy array and dispatching to the compiled kernel is therefore amortised over more work at larger N, making the relative advantage of NumPy greater.

Second, at small N (e.g. $N = 100$) the interaction lists are short—typically only a handful of nodes per particle—and the per-call Python overhead of `np.array()` and the NumPy dispatcher is comparable to the actual arithmetic. This explains why the speedup at $N = 100$ is modest. By $N = 2000$ the lists are long enough that the compiled arithmetic dominates, and the speedup stabilises near 2×.

It is worth noting that the outer loop over all N particles remains a Python `for` loop in both versions. The speedup reported here reflects only the vectorisation of the inner force kernel; a further improvement would come from parallelising the outer loop across particles, which was beyond the scope of this milestone.

2.5 Mapping to RISC-V RVV Assembly

The NumPy kernel maps closely to a RISC-V Vector Extension (RVV) implementation. The correspondence is described below, following the standard strip-mining idiom used in earlier milestones.

Data layout and setup

Before the strip loop begins, the interaction list for particle i must be available as a contiguous array of 32-bit node indices in memory. In the Python implementation this is a Python list converted to an `int32` NumPy array; in assembly it would be written to a scratchpad buffer by the scalar traversal phase. The node field arrays (`node_com_x`, `node_com_y`, `node_com_z`, `node_mass`) are already contiguous `float32` arrays in the node pool, making them directly addressable by indexed loads.

Strip-mining loop

```
# Register assignments:
# a0 = pointer to interaction list (int32[])
# a1 = remaining list length
# a2 = base of node_com_x (float32[])
# a3 = base of node_com_y
# a4 = base of node_com_z
# a5 = base of node_mass
# fa0/fa1/fa2 = px[i], py[i], pz[i]  (scalars, broadcast via vfrsub.vf)
# fa3/fa4/fa5 = ax/ay/az accumulators (initialised to 0.0 before loop)
# fa6 = softening^2  (pre-computed)
# fa7 = G            (gravitational constant)
# ft0 = 3.0,  ft1 = 0.5  (Newton-Raphson constants)

strip_loop:
    vsetvli  t0, a1, e32, m1, ta, ma  # VL = min(a1, VLEN/32); e32 =
       float32

    # Step 1: load node indices for this strip
    vle32.v   v0, (a0)                # v0 = ilist[0..VL-1]  (int32)
    vsll.vi   v1, v0, 2               # v1 = byte offsets = index * 4

    # Step 2: gather node data using indexed loads  (vluxei32)
    vluxei32.v  v2, (a2), v1          # v2 = node_com_x[ilist]
    vluxei32.v  v3, (a3), v1          # v3 = node_com_y[ilist]
    vluxei32.v  v4, (a4), v1          # v4 = node_com_z[ilist]
    vluxei32.v  v5, (a5), v1          # v5 = node_mass[ilist]

    # Step 3: displacement vectors  dx = com - p[i]
    vfrsub.vf   v2, v2, fa0           # v2 = node_com_x - px[i]
    vfrsub.vf   v3, v3, fa1           # v3 = node_com_y - py[i]
    vfrsub.vf   v4, v4, fa2           # v4 = node_com_z - pz[i]

    # Step 4: softened distance squared
    vfmul.vv    v6, v2, v2            # dx*dx
    vfmacc.vv   v6, v3, v3            # += dy*dy
    vfmacc.vv   v6, v4, v4            # += dz*dz
    vfadd.vf    v6, v6, fa6           # += softening^2

    # Step 5: inv_dist3 = dist_sq^{-1.5}
    #         RVV has no direct ^{-1.5}; use rsqrt7 + Newton-Raphson
    vfrsqrt7.v  v7, v6                # v7 ~ 1/sqrt(dist_sq)  (7-bit
       approx)
    vfmul.vv    v8, v7, v7            # v8 = approx^2
    vfmul.vv    v8, v8, v6            # v8 = dist_sq * approx^2
    vfrsub.vf   v8, v8, ft0           # v8 = 3 - dist_sq*approx^2
    vfmul.vf    v8, v8, ft1           # v8 *= 0.5
    vfmul.vv    v7, v7, v8            # v7 = refined rsqrt ~ 1/sqrt(
       dist_sq)
    vfmul.vv    v9, v7, v7            # v9 = 1/dist_sq
    vfmul.vv    v7, v9, v7            # v7 = inv_dist3 = dist_sq^{-1.5}

    # Step 6: force scalar per node:  f = G * mass * inv_dist3
```

```
    vfmul.vf    v5, v5, fa7           # v5 *= G
    vfmul.vv    v5, v5, v7            # v5 = f[k] for each k in strip

    # Step 7: accumulate ax += sum(f * dx)
    vfmul.vv    v9, v5, v2            # f * dx
    vfredusum.vs v10, v9, v31         # horizontal sum -> v10[0]
    vfmv.f.s    ft2, v10              # extract scalar
    fadd.s      fa3, fa3, ft2         # ax accumulator

    vfmul.vv    v9, v5, v3            # f * dy
    vfredusum.vs v10, v9, v31
    vfmv.f.s    ft2, v10
    fadd.s      fa4, fa4, ft2         # ay accumulator

    vfmul.vv    v9, v5, v4            # f * dz
    vfredusum.vs v10, v9, v31
    vfmv.f.s    ft2, v10
    fadd.s      fa5, fa5, ft2         # az accumulator

    # Step 8: advance pointer and loop
    slli        t1, t0, 2             # bytes consumed = VL * 4
    add         a0, a0, t1            # advance index pointer
    sub         a1, a1, t0            # remaining -= VL
    bnez        a1, strip_loop
```

Listing 4: RVV pseudo-assembly for the vectorised force kernel

Correspondence between NumPy and RVV

Each line of the NumPy kernel maps directly to a block of RVV instructions:

- `ncx_np[ilist_np]` → `vluxei32.v`. The interaction-list indices serve as the offset array for an indexed (gather) load. This is exactly the use case `vluxei32` is designed for: loading non-contiguous elements from a base array using a vector of byte offsets.
- `dx*dx + dy*dy + dz*dz + soft*soft` → `vfmul.vv` and `vfmacc.vv`. The fused multiply-accumulate instruction `vfmacc.vv` computes $v_{\text{dst}} \mathrel{+}= v_a \times v_b$ in a single instruction, matching the compact NumPy expression and avoiding an extra temporary register.
- `dist_sq ** -1.5` → `vfrsqrt7.v` plus Newton-Raphson refinement. RISC-V RVV does not provide a direct reciprocal-to-the-1.5-power instruction. The standard approach uses the 7-bit reciprocal square-root approximation (`vfrsqrt7.v`) followed by one Newton-Raphson iteration to recover sufficient precision, then a final multiply to produce $(\sqrt{d^2})^{-3} = (d^2)^{-3/2}$.
- `np.sum(f * dx)` → `vfmul.vv` followed by `vfredusum.vs`. The reduction instruction writes the horizontal sum into element 0 of a destination vector register. `vfmv.f.s` then extracts that scalar, which is added to the running accumulator with `fadd.s`.

The strip-mining structure is identical to that developed in Milestone 3: `vsetvli` at the top of every iteration sets the active vector length to at most $\lfloor \text{VLEN}/32 \rfloor$ elements, advancing by that amount each iteration until the list is exhausted. The `ta` (tail agnostic) policy in `vsetvli` ensures that the final partial strip is handled correctly without any special-case code for the remainder.

The one element that distinguishes this kernel from those in Milestone 3 is the indexed gather (`vluxei32.v`). The force kernel in Milestone 3 operated on contiguous arrays and could use unit-stride loads (`vle32.v`). Here the interaction list contains arbitrary node indices, so a gather is unavoidable. This is precisely the instruction identified in the Milestone 4 specification as the key building block for this problem, and it maps directly to what `ncx_np[ilist_np]` does in NumPy.

3 Manal Fatima (29018) — Approach and Implementation

3.1 Research Summary

The implementation was based on two papers. Both solve the same problem of vectorising treecodes, but they do so in different ways and together they explained the design decisions set down throughout this document.

Barnes and Hut (1986). – A Hierarchical O(n log n) Force-Calculation Algorithm. This is the original paper containing the description of the Barnes-Hut algorithm [1]. Reading it was essential to the understanding of what the force calculation involves in the first place, before any extra vectorisation is attempted at. The main idea of the paper is to use an octree structure to organize distant particles so that one doesn't need to perform direct $O(N^2)$ pairwise computations. This algorithm traverses the tree from the root and at each level applies the opening criterion, which is s/d ¡ (where s is the size of the node, and d is the distance from the node to its centre of mass (CoM)). If it is satisfied, the entire node is considered a point mass, and if it is not satisfied, the node is opened and the process is repeated for its children.

One important point that this paper highlighted, and that I think is overlooked, is the nature of the vectorisation challenge. The path a particle takes through the tree is sequentially and data-dependent: A particle's path through the tree varies completely according its position to each node; different particles' paths through the tree vary completely. The force calculation itself (after identifying the interacting nodes) is straightforward, with no branching. This distinction implies the design below: vectorise the force kernel, rather than the traversal.

Dehnen (2002). – The algorithm of dehnen (2002) is called a Hierarchical O(N) Force Calculation Algorithm. The approach taken here was more directly applicable to Dehnen's paper [2]. Dehnen's central concept is that of "mutual cell-cell interactions". The standard Barnes-Hut algorithm works as follows: each particle walks the tree independently of one another. This is inefficient because particles that are close together in space traverse a lot of the same tree nodes in their walks, notes Dehnen. When processing a bunch of particles against a node all at once, it would do the same arithmetic less times.

A key contribution of Dehnen's is the dual-tree traversal where two trees, one a source tree and the other a sink tree, are traversed in the same step. If two nodes (1 from each tree) are sufficiently separated such that the force between them and any individual particle satisfies the "Barnes-Hut" criteria, then the force between each of these two nodes and a single particle can be calculated in a single "batched operation. This insight directly affect the approach used here, the BFS approach: Here particles are not processed one by one, but processed all at once at each level of the tree.

3.1.1 Comparison of Scalar and Vectorised Kernels

Scalar kernel (Milestone 3 reference). The code below shows the original scalar implementation. It processes one particle at a time, accumulating forces recursively during the tree walk.

```
def calculate_force_bh(body_idx, node_idx, theta_val):
    if node_mass[node_idx] == 0.0:
        return
    if node_is_leaf[node_idx] and node_particle[node_idx] == body_idx:
        return  # skip self-interaction

    dx = node_com_x[node_idx] - px[body_idx]
    dy = node_com_y[node_idx] - py[body_idx]
    dz = node_com_z[node_idx] - pz[body_idx]
```

```
    r2 = dx*dx + dy*dy + dz*dz
    r = math.sqrt(r2)

    if node_is_leaf[node_idx] or (r > 0.0 and node_size[node_idx] / r <
        theta_val):
         r_soft = math.sqrt(r2 + softening * softening)
         r_cubed = r_soft * r_soft * r_soft
         factor = G * node_mass[node_idx] / r_cubed
         ax[body_idx] += factor * dx
         ay[body_idx] += factor * dy
         az[body_idx] += factor * dz
    else:
         for k in range(8):
              child = node_child[node_idx * 8 + k]
              if child != -1:
                   calculate_force_bh(body_idx, child, theta_val)
```

Listing 5: Scalar force accumulation during tree traversal (Milestone 3)

BFS vectorised kernel. The code below shows the outer BFS traversal that processes all particles simultaneously. The queue holds tuples of (node_idx, particle_indices), where particle_indices is a NumPy array of indices for particles that still need to be processed against this node.

```
global np_ax, np_ay, np_az

    # zero accelerations
    np_ax[:] = 0.0
    np_ay[:] = 0.0
    np_az[:] = 0.0

    all_particles = np.arange(n, dtype=np.int32)
    queue = [(root, all_particles)]

    while len(queue) > 0:
         next_queue = []

         for node_idx, particle_indices in queue:

              # skip empty nodes
              if np_node_mass[node_idx] == 0.0:
                   continue

              # skip if no particles to process
              if len(particle_indices) == 0:
                   continue

              # get positions of all particles in this batch
              bx = np_px[particle_indices]
              by = np_py[particle_indices]
              bz = np_pz[particle_indices]

              # vector from each particle to node centre of mass
              dx = np_node_com_x[node_idx] - bx    # shape (k,)
              dy = np_node_com_y[node_idx] - by
              dz = np_node_com_z[node_idx] - bz
```

```
33
34              # distance for all k particles at once
35              r2 = dx*dx + dy*dy + dz*dz           # shape (k,)
36              r  = np.sqrt(r2)                      # shape (k,)
37
38              s = np_node_size[node_idx]
39              criterion = (r > 0.0) & (s / (r + 1e-30) < theta)   # shape (k
                    ,) boolean
40
41              # handle leaf nodes - must apply force directly (no children)
42              is_leaf = np_node_is_leaf[node_idx] == 1
43
44              if is_leaf:
45                  # leaf: apply force to all particles except self
46                  leaf_particle = np_node_particle[node_idx]
47                  if leaf_particle >= 0:
48                      # remove self-interaction: mask out the particle that
                            lives here
49                      not_self = particle_indices != leaf_particle
50                      apply_force_vectorized(node_idx, particle_indices[
                            not_self])
51                  else:
52                      apply_force_vectorized(node_idx, particle_indices)
53                  continue
54
55
56              close_enough = particle_indices[criterion]
57              if len(close_enough) > 0:
58                  apply_force_vectorized(node_idx, close_enough)
59
60              # particles where criterion is False -> node too close -> open
                    children
61              need_to_open = particle_indices[~criterion]
62              if len(need_to_open) > 0:
63                  for k in range(8):
64                      child = int(np_node_child[node_idx * 8 + k])
65                      if child != -1 and np_node_mass[child] > 0.0:
66                          next_queue.append((child, need_to_open))
67
68          queue = next_queue
69
70      # copy results back to python lists for leapfrog integrator
71      for i in range(n):
72          ax[i] = float(np_ax[i])
73          ay[i] = float(np_ay[i])
74          az[i] = float(np_az[i])
```

Listing 6: BFS traversal processing

Vectorised force kernel. The code below shows the inner kernel that computes force contributions from a single node to a batch of particles. This function contains no Python loops over particles; all operations are NumPy array operations that execute in compiled C code.

```
1  def apply_force_vectorized(node_idx, particle_indices):
2      # Gather positions of all particles in this batch
```

```
    bx = np_px[particle_indices]      # shape (k,)
    by = np_py[particle_indices]
    bz = np_pz[particle_indices]

    # Displacement vectors - node COM broadcasts to all particles
    dx = np_node_com_x[node_idx] - bx
    dy = np_node_com_y[node_idx] - by
    dz = np_node_com_z[node_idx] - bz

    # Softened distance squared and cubed - all k particles at once
    r2 = dx*dx + dy*dy + dz*dz + softening*softening   # shape (k,)
    r_soft = np.sqrt(r2)                               # shape (k,)
    r_cubed = r2 * r_soft                              # shape (k,)

    # Gravitational factor for all k particles at once
    factor = G * np_node_mass[node_idx] / r_cubed       # shape (k,)

    # Accumulate force contributions - no Python loop
    np_ax[particle_indices] += factor * dx
    np_ay[particle_indices] += factor * dy
    np_az[particle_indices] += factor * dz
```

Listing 7: Vectorised force kernel – one node against a batch of particles

3.1.2 Key Differences and Their Performance Implications

Three specific changes account for the performance difference between the scalar and BFS vectorised versions.

Change 1: Traversal structure. The scalar version runs one depth-first tree walk per particle. For N particles and a tree of depth approximately $\log_8(N)$, this requires roughly $N \times$ depth independent recursive walks. The BFS version runs one pass per tree level. At each level, there is a queue of nodes to process, and every particle is tested against each node in the queue using a single NumPy comparison. The number of Python-level loop iterations drops from $N \times$ depth to approximately depth passes through the outer while loop.

Change 2: Batched criterion evaluation. In the scalar code, `node_size[node_idx]`$/r < \theta$ is a single Python scalar comparison executed once per (particle, node) pair. In the BFS version, the equivalent line is:

```
criterion = (r > 0.0) & (s / (r + 1e-30) < theta)
```

Here, r is a NumPy array of shape $(k,)$ containing distances for all k particles in the current batch. NumPy evaluates the entire comparison as a single compiled C operation, producing a boolean array of shape $(k,)$. What was k separate Python comparisons becomes one operation with no Python interpreter overhead per element.

Change 3: Vectorised force kernel. In the scalar code, each force contribution is accumulated one node at a time inside the recursion. In the BFS version, `apply_force_vectorized` receives a NumPy array of particle indices and computes the force from one node to all those particles simultaneously. The scalar node centre-of-mass value broadcasts automatically across the array of particle positions. `np.sqrt` operates on all k distances at once. The accumulation via indexed assignment writes to the global acceleration arrays without any Python loop.

3.1.3 Connection to RVV Strip-Mining

The BFS approach naturally produces variable-length particle batches at each queue entry. Some nodes may have all N particles still pending, while deeper tree nodes may have only a handful. This variable-length batch corresponds directly to the strip-mining idiom supported by RISC-V RVV. The `vsetvli` instruction sets the active vector length to min(remaining, VLEN/32) elements at the top of each strip, processes that strip, then decrements the counter and loops. For each (node, batch) pair, an assembly implementation would strip-mine through the particle indices using `vsetvli`, processing as many as the vector unit can hold per iteration. This correspondence is elaborated further in Section 5.

3.2 Correctness Testing

Correctness was verified through three complementary approaches: automated tree structure tests, direct numerical comparison between scalar and vectorised force calculations, and conservation checks over multi-step simulations.

3.2.1 Tree Structure Tests

Four tests were executed automatically on the 100-body dataset before any simulation output was produced. These confirm that the tree is built correctly before the vectorised force calculation executes.

Test 1: Mass conservation. The root node's accumulated mass must equal the direct sum of all particle masses. The test computed both values and asserted their absolute difference was less than 10^{-3} times the total. This passed with root mass matching the particle sum to within floating-point rounding, confirming that `compute_mass` propagates masses correctly from leaves to root.

Test 2: All particles found. A depth-first traversal from the root must locate every particle index exactly once. This verifies that `insert_particle` correctly places every body into the octree regardless of its spatial position, and that no particle is lost or duplicated during subdivision. This passed for all 100 bodies.

Test 3: Centre-of-mass calculation. The tree-computed centre of mass matched the direct weighted average to full double precision on all three axes:

$$x_{\text{cm}} = 5.670615 \times 10^{7} \text{ m},$$
$$y_{\text{cm}} = 1.159580 \times 10^{7} \text{ m},$$
$$z_{\text{cm}} = -1.031701 \times 10^{7} \text{ m}.$$

Test 4: Bounding box containment. Every particle was confirmed to lie within the bounding box of its leaf node on all three axes, verifying that `subdivide` assigns child extents consistently with `get_octant`.

3.2.2 Numerical Comparison: Scalar vs. BFS Vectorised

A correctness function ran both the scalar and BFS vectorised force calculations on the same tree and compared the results. For the 100-body dataset, the following was observed:

The maximum relative error of 3.357×10^{-16} is at the level of IEEE 754 double-precision machine epsilon ($\epsilon_{\text{mach}} \approx 2.22 \times 10^{-16}$). This indicates that the BFS vectorised implementation agrees with the scalar reference to the last representable bit. The small discrepancy that exists is not an error

Metric	Value
Maximum absolute error in acceleration	3.469×10^{-18} m/s²
Maximum acceleration magnitude	1.034×10^{-2} m/s²
Maximum relative error	3.357×10^{-16}

but an expected consequence of floating-point non-associativity: the scalar code accumulates force contributions in the order visited during DFS traversal, while NumPy's internal operations use a different summation order. Both orderings are mathematically correct; they simply round differently at the least significant bits.

3.2.3 Energy and Momentum Conservation

A full simulation of 100 time steps was run using the BFS vectorised force calculation. The leapfrog KDK integrator was used with $\theta = 0.5$. The results are summarised below.

Step	Total Energy (J)	Centre of Momentum (kg·m/s)
Initial	-3.960952×10^{34}	$(3.818 \times 10^{29}, -3.842 \times 10^{29}, -5.693 \times 10^{28})$
Step 10	-3.960952×10^{34}	$(3.818 \times 10^{29}, -3.842 \times 10^{29}, -5.693 \times 10^{28})$
Step 50	-3.960952×10^{34}	$(3.818 \times 10^{29}, -3.842 \times 10^{29}, -5.693 \times 10^{28})$
Step 100	-3.960952×10^{34}	$(3.817 \times 10^{29}, -3.842 \times 10^{29}, -5.693 \times 10^{28})$

Energy drift: -0.000004% over 100 steps. This is within the expected range for the leapfrog integrator with $\theta = 0.5$ and matches the scalar version's conservation properties.

Momentum conservation: The total momentum vector remained constant to within floating-point rounding throughout the simulation. This confirms that the vectorised force calculation does not introduce any asymmetric or spurious forces.

3.3 Performance Measurement

Wall-clock time was measured using `time.perf_counter` around the force calculation call only, excluding tree construction and diagnostic output. Each measurement is the average of three independent runs on the same machine. Table 1 presents the results.

Table 2: Force calculation wall-clock time: scalar vs. BFS vectorised

N	Scalar (s)	BFS NumPy (s)	Speedup
100	0.0203	0.0078	2.59×
500	0.2689	0.0525	5.12×
1000	0.7627	0.1274	5.99×
2000	1.9536	0.2304	8.48×

The speedup grows consistently from 2.14× at $N = 100$ to 7.84× at $N = 2000$. This trend can be explained by two factors.

At small N (100), the number of particles in each BFS batch is small, so the NumPy arrays being processed are short. The fixed overhead of constructing and executing a NumPy operation – memory allocation, dispatcher call, compiled kernel launch – is comparable to the actual arithmetic being performed. This explains why the speedup at $N = 100$ is modest relative to larger values.

At large N (2000), each BFS queue entry carries a larger batch of particles. When the queue processes a high-level node that has not yet been subdivided (because the Barnes-Hut criterion is satisfied for many particles simultaneously), `apply_force_vectorized` receives hundreds of particle indices at a time. NumPy executes the distance and force computation for all of them in a single compiled C call. The fixed overhead is amortised over significantly more work, and the speedup reflects the full benefit of moving arithmetic out of the Python interpreter and into compiled code.

It is worth noting that the outer structure of the BFS loop itself remains Python code. The `while` loop over BFS levels and the `for` loop over nodes in the queue are not vectorised. The speedup reported in Table 1 comes entirely from vectorising the force kernel and the Barnes-Hut criterion evaluation. Further speedup would be possible by parallelising the outer loop across particles using something like `multiprocessing` or by compiling the entire BFS traversal with Numba, but this was beyond the scope of the milestone.

3.4 Mapping to RISC-V RVV Assembly

The BFS vectorised kernel maps directly to a RISC-V Vector Extension (RVV) implementation. This section describes the correspondence, following the strip-mining idiom established in Milestone 3.

3.4.1 Data Layout

After the tree is built, the node field arrays `node_com_x`, `node_com_y`, `node_com_z`, and `node_mass` are contiguous in memory (Python lists converted to NumPy arrays). The BFS queue carries arrays of particle indices. In an assembly implementation, the node pool would reside in a contiguous memory region, and the particle batch for each queue entry would be an integer array of indices stored in a scratchpad buffer.

3.4.2 The Strip-Mining Loop

For a given node and batch of particles, the assembly kernel would process the particle batch in strips of size VLEN/32 using `vsetvli`. The following pseudo-assembly illustrates the structure:

```
# Register assignments:
# a0 = pointer to particle index array (int32[])
# a1 = remaining batch size
# a2 = base of np_px (float32[])
# a3 = base of np_py
# a4 = base of np_pz
# fa0 = node_com_x[node_idx] (scalar, pre-loaded)
# fa1 = node_com_y[node_idx]
# fa2 = node_com_z[node_idx]
# fa3 = G * node_mass[node_idx] (scalar, pre-computed)
# fa4 = softening^2 (scalar, pre-computed)
# a5 = base of np_ax (float32[])
# a6 = base of np_ay
# a7 = base of np_az

strip_loop:
    vsetvli t0, a1, e32, m1, ta, ma    # VL = min(a1, VLEN/32)

    # Step 1: Load particle indices for this strip
    vle32.v   v0, (a0)                 # v0 = indices[0..VL-1] (int32)
```

```
    vsll.vi   v1, v0, 2                # v1 = byte offsets = index * 4

    # Step 2: Gather particle positions using indexed loads
    vluxei32.v v2, (a2), v1           # v2 = np_px[indices]
    vluxei32.v v3, (a3), v1           # v3 = np_py[indices]
    vluxei32.v v4, (a4), v1           # v4 = np_pz[indices]

    # Step 3: dx = node_com_x - px[i] for all particles
    vfrsub.vf v2, v2, fa0             # v2 = node_com_x - px
    vfrsub.vf v3, v3, fa1             # v3 = node_com_y - py
    vfrsub.vf v4, v4, fa2             # v4 = node_com_z - pz

    # Step 4: Softened distance squared
    # r2 = dx*dx + dy*dy + dz*dz + softening^2
    vfmul.vv  v5, v2, v2              # dx*dx
    vfmacc.vv v5, v3, v3              # + dy*dy
    vfmacc.vv v5, v4, v4              # + dz*dz
    vfadd.vf  v5, v5, fa4             # + softening^2

    # Step 5: inv_dist3 = r2^{-1.5}
    # RVV has no direct -1.5 power; use rsqrt7 + Newton-Raphson
    vfrsqrt7.v v6, v5                 # v6 approx 1/sqrt(r2) (7-bit)
    vfmul.vv   v7, v6, v6             # v7 = approx^2
    vfmul.vv   v7, v7, v5             # v7 = r2 * approx^2
    vfrsub.vf  v7, v7, ft0            # v7 = 3 - r2*approx^2 (ft0=3.0)
    vfmul.vf   v7, v7, ft1            # v7 *= 0.5 (ft1=0.5)
    vfmul.vv   v6, v6, v7             # v6 = refined rsqrt
    vfmul.vv   v7, v6, v6             # v7 = 1/r2
    vfmul.vv   v6, v7, v6             # v6 = inv_dist3 = r2^{-1.5}

    # Step 6: factor = G * node_mass * inv_dist3
    vfmul.vf   v6, v6, fa3            # v6 = factor[k] for each k

    # Step 7: Gather existing accelerations and accumulate
    vluxei32.v v8, (a5), v1           # v8 = np_ax[indices]
    vfmacc.vv  v8, v6, v2             # np_ax += factor * dx
    vsuxei32.v v8, (a5), v1           # scatter store back

    vluxei32.v v8, (a6), v1           # v8 = np_ay[indices]
    vfmacc.vv  v8, v6, v3             # np_ay += factor * dy
    vsuxei32.v v8, (a6), v1

    vluxei32.v v8, (a7), v1           # v8 = np_az[indices]
    vfmacc.vv  v8, v6, v4             # np_az += factor * dz
    vsuxei32.v v8, (a7), v1

    # Step 8: Advance and loop
    slli t1, t0, 2                    # bytes = VL * 4
    add a0, a0, t1                    # advance index pointer
    sub a1, a1, t0                    # remaining -= VL
    bnez a1, strip_loop
```

Listing 8: RVV strip-mining loop for the force kernel

3.4.3 Correspondence Between NumPy Operations and RVV Instructions

Each line in the NumPy kernel maps to a specific block of RVV instructions:

NumPy Operation	RVV Instruction(s)	Explanation
`np_px[particle_indices]`	`vluxei32.v`	Gather load: loads n
`dx*dx + dy*dy + dz*dz`	`vfmul.vv` + `vfmacc.vv`	Fused multiply-accur
`np.sqrt(r2)`	`vfrsqrt7.v` + Newton-Raphson	Reciprocal square ro
`np_ax[particle_indices] += factor * dx`	`vluxei32.v` + `vfmacc.vv` + `vsuxei32.v`	Gather, multiply-acc
Loop control	`vsetvli` + branch	Sets active vector len

3.4.4 Key Difference from Milestone 3 Assembly

The force kernel in Milestone 3 operated on contiguous arrays and could use unit-stride loads (`vle32.v`). The BFS approach presented here requires indexed (gather) loads because each queue entry contains an arbitrary set of particle indices that are not contiguous in memory. `vluxei32.v` is the instruction designed for this exact pattern – it loads elements from a base address using offsets stored in a vector register. This maps directly to what `np_px[particle_indices]` does in NumPy.

3.5 Why Tree Construction Cannot Be Vectorised

For completeness, and to contrast with the force traversal phase, this section restates why the tree build phase cannot be vectorised using the same approach.

Tree construction is inherently sequential due to data-dependent control flow:

1. **Data-dependent decisions.** When inserting particle i into the tree, the octant it falls into depends on its position relative to the current node's midpoint. This decision cannot be made in advance and differs for every particle at every level of the tree.

2. **Index chaining.** Each step reads a child index from `node_child`, then uses that index to address the next node's data. This is a pointer-chasing pattern where each iteration depends on the result of the previous one.

3. **Read-after-write hazards.** Inserting one particle may trigger a subdivision that updates child pointers which the very next particle insertion must read. Processing multiple insertions simultaneously would introduce race conditions.

4. **Irregular memory access.** Nodes are allocated via a pool counter, so accesses are not stride-1 and cannot be expressed as a contiguous vector load.

By contrast, once the tree is built and the node arrays are fixed, the force calculation reads the tree but never writes to it. Different particles can be processed against the same node simultaneously with no conflicts. This is the fundamental reason why force calculation can be vectorised while tree construction cannot.

4 Areeba Salman (29075) — Approach and Implementation

4.1 Research Summary

To understand how the irregular data structures of a Barnes-Hut octree could be mapped to vector hardware, I researched the foundational literature on tree-code vectorization. A key paper influencing this implementation is *"A modified tree code: Don't laugh; it runs"* by Joshua E. Barnes (1990), alongside subsequent works analyzing Barnes-Hut on SIMD architectures.

The primary challenge outlined in the literature is that tree traversal is heavily data-dependent: every particle traverses a completely different path through the tree depending on its spatial location, making it impossible to predict memory access patterns. The proposed solution is to entirely decouple the sequential traversal phase from the mathematical force computation phase. In my implementation, a scalar processor performs the tree walk to construct an "interaction list" for each particle. Once this contiguous list of node indices is built, it is passed to the vector hardware (simulated here via NumPy).

4.2 Implementation Design

By decoupling the traversal from the math, the inner force kernel was completely flattened.

Scalar inner kernel (from Milestone 3):

```
def compute_force_scalar(i, interaction_list):
    ax = ay = az = 0.0
    for node in interaction_list:
        dx = node_cx[node] - px[i]
        dy = node_cy[node] - py[i]
        dz = node_cz[node] - pz[i]
        dist_sq = dx*dx + dy*dy + dz*dz + softening**2

        inv_dist3 = dist_sq ** -1.5
        f = G * node_mass[node] * inv_dist3

        ax += f * dx
        ay += f * dy
        az += f * dz
    return ax, ay, az
```

NumPy vectorised kernel:

```
import numpy as np

def compute_force_numpy(i, interaction_list):
    if not interaction_list:
        return 0.0, 0.0, 0.0

    # interaction_list is cast to a numpy array of node indices
    nodes = np.array(interaction_list, dtype=np.int32)

    # Vectorised gather indexing
    dx = node_cx[nodes] - px[i]
```

```
    dy = node_cy[nodes] - py[i]
    dz = node_cz[nodes] - pz[i]

    # Vectorised distance and reciprocal cube
    dist_sq = dx*dx + dy*dy + dz*dz + (softening**2)
    inv_dist3 = dist_sq ** -1.5

    # Element-wise force multiplication
    f = G * node_mass[nodes] * inv_dist3

    # Vectorised reduction
    ax = np.sum(f * dx)
    ay = np.sum(f * dy)
    az = np.sum(f * dz)

    return ax, ay, az
```

Key Observations:

- The `for` loop over the interaction list is eliminated entirely.
- `node_cx[nodes]` uses the interaction list to fetch the positions of all interacting nodes simultaneously using vectorized gather indexing.
- `dist_sq ** -1.5` issues a single vectorized `pow`/`rsqrt` call that the NumPy backend lowers to C-level SIMD instructions on the host CPU.
- `np.sum()` is utilized as a highly optimized vectorized reduction to accumulate the final forces.

4.3 Correctness Testing

To verify the correctness of the NumPy implementation against the scalar reference, I tracked the total system energy, center of mass, and total momentum across 100 time-steps for $N = 100$, 500, 1000, and 2000.

Taking $N = 500$ as a representative benchmark, both the scalar and NumPy implementations yielded an identical final total energy of -1.512468×10^{35} Joules, with an identical energy drift of 4.777968%. Furthermore, the Center of Mass at step 100 for both implementations remained perfectly synchronized at $(1.247 \times 10^8,\ -3.417 \times 10^7,\ 4.014 \times 10^7)$. Because the system's global physical invariants evolved and degraded at the exact same floating-point rate, the vectorized mathematical kernel is proven to be mathematically equivalent to the scalar reference.

4.4 Performance

N	Scalar (s)	NumPy (s)	Speedup
100	1.60	3.33	0.48x
500	16.09	31.54	0.51x
1000	46.96	81.08	0.58x
2000	118.22	180.31	0.66x

Performance Analysis & Overhead Caveat:

As shown in the table, the NumPy-accelerated implementation is paradoxically slower than the pure scalar Python implementation, yielding a speedup of less than 1.0x across all tested values of N. However, the speedup ratio slowly improves as N scales higher (climbing from 0.48x at $N = 100$ to 0.66x at $N = 2000$).

This performance inversion highlights a critical constraint of modeling hardware vectorization in a high-level interpreted language. In our Python implementation, the interaction list is built dynamically as a standard list, and then explicitly cast into a contiguous NumPy array (`np.array(...)`) so it can be used as a gather index. Because this casting and memory-allocation operation occurs inside the main force loop—executed N times per timestep—the overhead of transitioning data between the Python interpreter and the C-backend completely eclipses the time saved by the SIMD arithmetic. As N grows, the interaction list grows, and the vector math bears more computational weight, mitigating the casting penalty slightly. In a true RISC-V RVV environment, this overhead would not exist, as the interaction list would remain in cache and be passed directly to the vector load instructions.

4.5 Mapping to RISC-V RVV Assembly

The NumPy implementation serves as a direct, testable model for how the RISC-V Vector Extension (RVV) would execute the Barnes-Hut force calculation:

- **Gather Loads (`vluxei32.v`):** The NumPy operation `dx = node_cx[nodes] - px[i]` perfectly mirrors the RVV indexed gather load instruction (`vluxei32.v`). Because the nodes in the interaction list are scattered randomly throughout the pool memory, stride-1 vector loads (`vle32.v`) cannot be used. RVV uses the interaction list array as an index vector to gather the x, y, z coordinates into vector registers (`v1`, `v2`, `v3`).
- **Vector Math (`vfsub.vv`, `vfmacc.vv`):** The element-wise distance and force calculations map directly to RVV vector-vector arithmetic. For example, `dx*dx + dy*dy + dz*dz` maps to a sequence of vector multiply-accumulate instructions (`vfmacc.vv`) operating on `VLEN/32` elements simultaneously.
- **Strip-Mining:** Our Python code naturally processes arrays of arbitrary lengths. In assembly, this would be handled using standard strip-mining. The `vsetvli` instruction would dynamically set the active vector length (`vl`) based on the remaining number of nodes in the interaction list for particle i, ensuring the loop handles varying list sizes without out-of-bounds memory accesses.
- **Vector Reduction (`vfredusum.vs`):** Finally, the NumPy `np.sum(f * dx)` operation directly corresponds to the `vfredusum.vs` instruction. After processing the interaction list for a single particle, the vector register containing the force components is reduced into a single scalar float accumulator register (`fadd.s`), yielding the final acceleration for particle i.

References

[1] Makino, J. (1990). Vectorization of a treecode. *Journal of Computational Physics*, **87**(1), 148–160.

[2] Warren, M. S. & Salmon, J. K. (1993). A parallel hashed oct-tree N-body algorithm. In *Proceedings of Supercomputing '93*, pp. 12–21. ACM Press.

[3] Barnes, J. and Hut, P. (1986). A hierarchical O(N log N) force-calculation algorithm. *Nature*, 324(4), 446-449. `https://doi.org/10.1038/324446a0`

[4] Dehnen, W. (2002). A hierarchical O(N) force calculation algorithm. *Journal of Computational Physics*, 179, 27-42. `https://arxiv.org/pdf/astro-ph/0202512`